Introduction to brfssTools

Working with longitudinal survey data is historically tedious. Variable names change, value codes shift (e.g., “1” means “Yes” in 2015, but “2” means “Yes” in 2016), and question text drifts.

The `brfssTools` package solves this by providing a unified harmonization engine for Oregon BRFSS data. Because BRFSS microdata is restricted, this package does not include the raw data itself. Instead, it uses a **Bring Your Own Data (BYOD)** approach, allowing you to seamlessly query, load, and harmonize your securely stored files.

This vignette covers the standard workflow: setting up your data pool, finding variables, and extracting datasets.

1. Setting Up Your Data Pool

Before pulling data, you must tell `brfssTools` where your raw data lives.

Your data pool should be a single directory containing the yearly Oregon BRFSS data files saved as `.csv` files (e.g., `BRFSS_2018.csv`, `BRFSS_2019.csv`).

Set your pool directory using `set_survey_pool()`:

```r
library(brfssTools)

# Point to the secure local or network drive containing the .csv files
set_survey_pool("Z:/Secure_Data/BRFSS/Harmonized_Pool")
```

Note: You only need to run this once per session. The package will remember this location in the background for all subsequent data pulls.

2. Loading and Searching the Crosswalk

The core of `brfssTools` is its crosswalk: a meticulously curated map of how raw variables across different years translate into unified “concepts.”

First, load the crosswalk into memory. By default, `load_crosswalk()` loads all available years. If you only want to analyze specific years, you can pass them as an argument to save memory and speed up processing:

```r
# Load the BRFSS crosswalk map for all years
cw <- load_crosswalk()

# Or, load the crosswalk strictly for 2018 and 2019
cw_subset <- load_crosswalk(years = c(2018, 2019))
```

If you want to know if a topic is covered in the crosswalk, use `cw_search()`. You can search by regular expressions, and the function will scan concept labels, notes, and raw question text.

```r
# Search for anything related to asthma
cw_search(cw, "asthma")
```

```
#> # A tibble: 12 × 6
#>    survey  year raw_var_name question_text                matched_in concept_id
#>    <chr>  <int> <chr>        <chr>                        <chr>      <chr>
#>  1 BRFSS   2018 ASTHMA3      Ever been told you had asth… question   asthma_ever
#>  2 BRFSS   2018 ASTHNOW      Do you still have asthma?    question   asthma_curr
#>  3 BRFSS   2019 ASTHMA3      Ever been told you had asth… question   asthma_ever
#> # ℹ 9 more rows
```

3. Examining a Specific Concept

Once you identify a `concept_id` you want to analyze, you can see exactly how it was harmonized using `cw_lookup()`. This is crucial for transparency, allowing you to review how the underlying variable evolved over time.

```
cw_lookup(cw, "asthma_ever")
```

```
#> # A tibble: 5 × 11
#>   concept_id  survey  year raw_var_name recode_type recode_rule question_text
#>   <chr>       <chr>  <int> <chr>        <chr>       <chr>       <chr>
#> 1 asthma_ever BRFSS   2018 ASTHMA3      inline      1=1; 2=0    Ever been told…
#> 2 asthma_ever BRFSS   2019 ASTHMA3      inline      1=1; 2=0    Ever been told…
#> # ℹ ... additional columns for domain, missing values, and notes
```

In this example, we can see that the raw variable `ASTHMA3` codes “2” as “No”. The `recode_rule` tells the engine to convert “2” to “0” to create a standard binary indicator.

4. Pulling Data

The `cw_pull()` function is the workhorse of the package. You simply pass it the loaded crosswalk (`cw`), the concepts you want, and any identifier columns you need to keep (like respondent IDs or survey weights).

Because you already set your survey pool in Step 1, you do not need to manually load your data. The package will silently fetch those `.csv` files from your secure folder, process them, and return a long-format dataset.

Extracting Harmonized Data (Default)

By default, the package applies the recode rules to give you a clean, unified dataset.

```
# Pull harmonized asthma data, retaining the respondent sequence number and weight
asthma_data <- cw_pull(
  cw = cw,
  concept_ids = c("asthma_ever", "asthma_curr"),
  id_cols = c("SEQNO", "_LLCPWT")
)

head(asthma_data)
```

```
#> # A tibble: 6 × 7
#>   concept_id  survey  year raw_var_name SEQNO      _LLCPWT harmonized_value
#>   <chr>       <chr>  <int> <chr>        <chr>        <dbl> <chr>
#> 1 asthma_ever BRFSS   2018 ASTHMA3      2018000001   104.  0
#> 2 asthma_ever BRFSS   2018 ASTHMA3      2018000002   210.  1
#> 3 asthma_curr BRFSS   2018 ASTHNOW      2018000001   104.  NA
```

Note: `brfssTools` automatically handles survey skip logic by recognizing "missing" codes (e.g., blank, Refused, Don't Know) from the codebooks and coercing them to `NA`.

Extracting Raw Data Only

If you do not want the engine to alter your data and just want an easy way to pull the raw, un-recoded variables across years, set `harmonize = FALSE`.

```
asthma_raw <- cw_pull(
  cw = cw,
  concept_ids = c("asthma_ever"),
  id_cols = c("SEQNO"),
  harmonize = FALSE
)

head(asthma_raw)
```

```
#> # A tibble: 2 × 6
#>   concept_id  survey  year raw_var_name SEQNO      raw_value
#>   <chr>       <chr>  <int> <chr>        <chr>      <chr>
#> 1 asthma_ever BRFSS   2018 ASTHMA3      2018000001 2
#> 2 asthma_ever BRFSS   2018 ASTHMA3      2018000002 1
```

(You can also set `keep_raw = TRUE` and `harmonize = TRUE` to get both columns side-by-side to verify the recoding logic yourself).